Day 2

Enhancements of C in C++

Input and Output Enhancements

cin and cout – iostream Header

C++ replaces the traditional scanf() and printf() of C with cin and cout.

```
#include<iostream>
using namespace std;

int main()
{
        char name[20];
        int age;
        cout << "Enter name and age" << endl;
        cin >> name >> age;
        cout << name << "\t" << age << endl;
}
```

Key Points:

- cin and cout are objects of **istream** and **ostream** classes respectively.
- No **"&"** and **format specifiers** (%d, %s, etc.) are required.
- cin uses **>> (extraction operator)**.
- cout uses **<< (insertion operator)**.
- scanf() and printf() are **variable number of arguments functions**.

Clearing the Input Buffer

Problem Example:

```
#include <iostream>
using namespace std;

int main()
{
  int age;
  char name[20];
  cout << "Enter age" << endl;
  cin >> age;       // Suppose user types: 25 and presses enter
  cout << "Enter full name" << endl;
  cin.getline(name, 19); // Skips input due to leftover '\n'
  cout << name << "\t" << age << endl;
}
```

Explanation:

- After **cin >> age,** the newline '\n' from pressing Enter remains in the input buffer.
- The next **cin.getline()** immediately reads this newline as an empty line.

Solution – cin.ignore()

```
#include <iostream>
#include <limits>
using namespace std;

int main()
{
  int age;
  char name[20];
  cout << "Enter age" << endl;
  cin >> age;
  cout << "Enter full name" << endl;
  cin.ignore(numeric_limits<streamsize>::max(), '\n'); // Clears leftover '\n'
  cin.getline(name, 19);
  cout << name << "\t" << age << endl;
}
```

Explanation:

- **cin.ignore()** tells the input stream to **ignore characters** in the buffer.
- **numeric_limits<streamsize>::max**() specifies the maximum characters to ignore (effectively "infinite").
- **'\n'** tells it to stop ignoring after reaching a newline.

Function Declaration Requirement

In C++, **functions must be declared before use** (unlike C where implicit declarations were allowed).

```
#include<iostream>
using namespace std;

int main()
{
        disp(); // Error: disp not declared
        return 0;
}
void disp()
{
        cout << "in disp" << endl;
}
```

Name Mangling (Name Decoration)

Definition:
C++ compiler changes (mangles) the names of functions during compilation to support **function overloading**.

Example:

```
#include<iostream>
using namespace std;

void open(char* accno)
{
        cout << "Account Opened\t" << accno << endl;
}
void open(char* accno, double balance)
{
        cout << "Account Opened\t" << accno << "\t" << balance << endl;
}
```

```
int main()
{
        open("A100");
        open("A101", 10000);
        return 0;
}
```

Explanation:

- Different versions of open() are internally renamed differently depending on argument type, order, and number.
- This process is **compiler dependent** and is called **name mangling** or **name decoration**.

Dynamic Memory Management

a) new Operator

- Similar to **malloc()** or **calloc().**
- Allocates memory dynamically from **heap (free store)**.
- **Operator**, not a function.
- No need to use **sizeof.**
- Invokes **constructor** (if applicable).

b) delete Operator

- Similar to **free().**
- Deallocates memory allocated by new.
- **Operator**, not a function.
- Invokes **destructor**.

Example:

```
#include<iostream>
using namespace std;

int main()
{
    int* ptr, rec, i;
    cout << "\nHow many numbers ?\n";
    cin >> rec;
    ptr = new int[rec]; // dynamic allocation

    cout << "\nEnter " << rec << " numbers\n";
    for (i = 0; i < rec; i++)
            cin >> ptr[i];

    cout << "\nDisplaying all numbers\n";
    for (i = 0; i < rec; i++)
            cout << ptr[i] << endl;

    delete[] ptr; // correct deallocation for arrays
    return 0;
}
```

Scope Resolution Operator (::)

Used to access **global variables** or **class members** when local variables hide them.

```
#include<iostream>
using namespace std;

int val = 100; // global

int main()
{
	int val = 20; // local
	cout << val << endl;   // local value
	cout << ::val << endl;  // global value
}
```

Default Arguments

- C++ allows functions to have **default values** for parameters.
- These default values are used when the function is called without arguments.

```
void display(int x = 10, int y = 20)
{
	cout << x << "\t" << y << endl;
}

int main()
{
	display();      // uses default 10, 20
	display(5);     // uses 5, 20
	display(1, 2);   // uses 1, 2
}
```

Macros in C++

- Still supported through the **preprocessor**.
- Declared using #define.

```
#define PI 3.14
#define SQUARE(x) (x*x)
```

However, macros are **not type-safe** and **error-prone**, hence **inline functions are preferred** in C++.

Inline Functions

Definition:

An **inline function** is a function expanded **inline** when invoked — reducing the overhead of function calls.

```
inline void disp()
{
	cout << "Hello Inline" << endl;
}
```

Compiler Decision on Inline Functions

- The compiler may **internally treat an inline function like a macro**, depending on certain situations.

- Situations where the compiler may **not** make a function inline include:
    - If the function contains **many lines of code**.
    - If the function contains **recursive calls**.
- A **macro** works by **replacing the function call with the actual code** of the macro at compile time.
    - For each call, a **new copy of the macro code** is created.
    - If the macro contains **multiple lines**, it leads to **increased memory usage** since the code is duplicated at every call.
- To overcome these issues, **inline functions** were introduced in C++.
- An **inline function** is a **request to the compiler** to expand the function code inline where appropriate.
- The compiler decides whether to make the function inline, thereby **balancing execution speed and memory efficiency**.

Inline vs Macros

Aspect	Inline Function	Macro
Definition	A function whose code may be expanded **inline by the compiler** at the point of call. It is a **request** to the compiler to replace the call with the actual function code.	A **preprocessor directive** that performs **text substitution** before compilation. The call is replaced literally with the macro code.
Checked by Compiler	Fully **checked by the compiler** for syntax and type correctness.	**Not checked** by the compiler because macros are expanded during preprocessing.
Error Detection	**Compile-time errors** are detected during normal function compilation.	Errors may appear **after macro expansion**, making debugging difficult.
Scope	Follows **scope rules** of C++ (can exist inside classes, namespaces, etc.).	Has **no scope**; globally substituted wherever defined.
Memory and Performance	The compiler decides whether to inline based on factors like function size, recursion, and performance trade-offs — aims to **balance speed and memory efficiency**.	For every macro call, the **entire macro body is copied**, which can increase **code size and memory usage**.
Type Safety	Type checking is enforced; parameters have **defined types**.	**No type safety** — simple text substitution without validation.

---

1. Pointer to Constant

Syntax

```
datatype const* ptr;
or
const datatype * ptr;
```

Meaning:

- Can point to **const as well as non-const** members.
- Pointer **can be changed**, but using the pointer you **cannot modify (Change the value) of** the member it points to.
- We can change the pointer **(As it is not the constant pointer)**

Example 1: With Non-Const Member

```
int num = 100;
const int* ptr;    // or int const *ptr;
ptr = &num;

cout << num << "\t" << *ptr << endl;
num = 200;
cout << num << "\t" << *ptr << endl;

// *ptr = 400;  // not allowed
ptr++;  // allowed
```

Example 2: With Const Member

```
const int var = 500;
const int* ptr1;   // or int const *ptr1;
ptr1 = &var;

cout << var << "\t" << *ptr1 << endl;

// var = 600;  // not allowed
// *ptr1 = 600; // not allowed
ptr1--;  // allowed
```

---

2. Constant Pointer

Syntax

```
datatype* const pointervar = address;
```

Meaning:

- Can point to **non-const members only**.
- Pointer **cannot be changed**, but using the pointer you **can modify** the member it points to.

Example

```
int num = 200;
int* const ptr1 = &num;

cout << num << "\t" << *ptr1 << endl;

num = 400;  // allowed
cout << num << "\t" << *ptr1 << endl;

// ptr1++;  // not allowed

*ptr1 = 600;  // allowed
cout << num << "\t" << *ptr1 << endl;

const int num1 = 250;
// int *const ptr2 = &num1;   // not allowed
```

---

3. Constant Pointer to a Constant

Syntax

```
const datatype* const pointervar = address;
or
datatype const* const pointervar = address;
```

Meaning:

- Can point to **const as well as non-const members**.
- Neither the pointer nor the member can be modified.
- It is like a Read Only Pointer.

Example 1: With Non-Const Member

```
int num = 100;
const int* const ptr1 = &num;

cout << num << "\t" << *ptr1 << endl;
// ptr1++;   // not allowed
// *ptr1 = 400; // not allowed

num = 300; // allowed
cout << num << "\t" << *ptr1 << endl;
```

Pointer Types

Type	Can Point To Const Member	Can Point To Non-Const Member	Pointer Modifiable	Pointed Value Modifiable
Pointer to Constant	✅	✅	✅	❌
Constant Pointer	❌	✅	❌	✅
Constant Pointer to a Constant	✅	✅	❌	❌

Reference in C++

- A **reference** is just another name (**alias**) for a variable.
- A **reference is not allocated any separate memory**.
- A **reference must be initialized** when declared.
- Once initialized, it **cannot refer to another referent**.
- A reference **gets dereferenced automatically**, unlike a pointer that requires explicit dereferencing using *.

Example

```
int num = 10;
int& ref = num;
cout << num << endl;
cout << ref << endl;
```

Explanation

- The compiler treats ref as an alias for num.
- Every use of ref is replaced by num in the generated machine code (after compilation).
- Hence, ref doesn't exist as a separate variable in memory.

When we write:

```
cout << ref << endl;
```

- The compiler replaces ref with num internally.
- The machine code reads from num's memory location and prints 10.
- No dereference operator is needed; this is handled automatically by the compiler.

Pointer VS Reference

Pointer	Reference
Pointers are allocated memory separately in the program.	A reference is just another name for a variable and is not allocated separate memory.
A pointer can be incremented or decremented (unless it is a constant pointer).	A reference, once initialized, cannot be changed to refer to another variable.
Array of pointers is possible.	Array of references is not allowed.

Program 1: Basic Reference Behavior

```
#include<iostream>
using namespace std;
int main()
{
	int num = 30;
	int num1 = 40;
	// int &rr; // reference must be initialized
	int& ref = num;   // reference must be initialized
	int& ref1 = ref;  // ref1 actually refers to num

	cout << num << endl;
	cout << ref << endl;  // reference gets automatically dereferenced

	ref = 0;
	cout << num << endl;
	cout << ref1 << endl;

	int a = 20, b = 30, c = 40;
	// int &ref2[3] = {a, b, c}; // array of reference is not allowed
	// cout << ref2[0] << endl << ref2[1] << endl << ref2[2] << endl;
}
```

Program 2: Call by Reference Using References

```
#include<iostream>
using namespace std;
int main()
{
	void swap(int&, int&);
	int num1, num2;
	cout << endl << "Enter two nos\n";
	cin >> num1 >> num2;
	cout << "\nBefore call\t" << num1 << "\t" << num2 << endl;
	swap(num1, num2);
	cout << "\nAfter call\t" << num1 << "\t" << num2 << endl;
}

void swap(int& a, int& b) // Saving memory here
{
	int c = a;
	a = b;
	b = c;
}
```

Program 3: Reference to an Array and Reference to a Function

```
#include<iostream>
using namespace std;
int main()
{
	int arr[3] = { 10, 20, 30 };
	int(&ref)[3] = arr;     // reference to an array
	void disp();
	void(&ref1)() = disp;    // reference to a function

	for (int i = 0; i < 3; i++)
	{
		cout << endl << ref[i] << endl;
	}
	ref1();
	// ref++;  // cannot increment because ref gets automatically dereferenced
}

void disp()
{
```

```
	cout << "in disp\n";
}
```

Program 4: Constant References and Reference to Pointer

```
#include<iostream>
using namespace std;
int main()
{
	int num = 20;
	const int& ref = num;
	cout << endl << ref;
	// ref++; // Error
	num++;
	cout << endl << ref;

	const int num1 = 30;
	// int &ref1 = num1; // Error
	const int& ref1 = num1;

	int a = 50, * ptr = &a;
	cout << endl << *ptr;
	int*& ref3 = ptr;  // Reference to a Pointer
	cout << endl << *ref3 << endl;

	ptr++;
	cout << endl << ptr << endl;
	cout << endl << ref3 << endl;
}
```

Program 5: Reference and Function Overloading (with Constants and Ambiguity)

Case 1: Constants as Arguments

```
/* 10 and 20 cannot be passed to int &a and int &b because they are constant */

#include<iostream>
using namespace std;
int main()
{
	void disp(int, int);
	void disp(int&, int&);
	disp(10, 20); // in value
}

void disp(int& a, int& b)
{
	cout << "in reference\t" << a << "\t" << b << endl;
}

void disp(int a, int b)
{
	cout << "in value\t" << a << "\t" << b << endl;
}
```

Case 2: Ambiguity with Variables

```
/* x and y can be passed to int &a and int &b as well as int a and int b, hence ambiguity error */

#include<iostream>
using namespace std;
int main()
{
	void disp(int, int);
```

```
        void disp(int&, int&);
        int x = 3, y = 8;
        disp(x, y); // ambiguity error
}

void disp(int& a, int& b)
{
        cout << "in reference\t" << a << "\t" << b << endl;
}

void disp(int a, int b)
{
        cout << "in value\t" << a << "\t" << b << endl;
}
```

Case 3: Ambiguity with const int &

```
/* x and y can be passed to const int &a and const int &b
as well as int a and int b, hence ambiguity error */

#include<iostream>
using namespace std;
int main()
{
        void disp(int, int);
        void disp(const int&, const int&);
        int x = 3, y = 8;
        disp(x, y);  // error
}

void disp(const int& a, const int& b)
{
        cout << "in reference\t" << a << "\t" << b << endl;
}

void disp(int a, int b)
{
        cout << "in value\t" << a << "\t" << b << endl;
}
```

Case 4: Ambiguity with const Arguments

```
/* x and y can be passed to const int &a and const int &b
as well as int a and int b, hence ambiguity error */

#include<iostream>
using namespace std;
int main()
{
        void disp(int, int);
        void disp(const int&, const int&);
        const int x = 3, y = 8;
        disp(x, y);   // ambiguity error
}

void disp(const int& a, const int& b)
{
        cout << "in reference\t" << a << "\t" << b << endl;
}

void disp(int a, int b)
{
        cout << "in value\t" << a << "\t" << b << endl;
}
```

Case 5: Ambiguity with Literal Constants

```
/* 10 and 20 can be passed to const int &a and const int &b
as well as int a and int b, hence ambiguity error */

#include<iostream>
using namespace std;
int main()
{
	void disp(int, int);
	void disp(const int&, const int&);
	disp(10, 20);   // ambiguity error
}

void disp(const int& a, const int& b)
{
	cout << "in reference\t" << a << "\t" << b << endl;
}

void disp(int a, int b)
{
	cout << "in value\t" << a << "\t" << b << endl;
}
```

Memory Leak

Definition

- A **memory leak** occurs when a pointer is pointing to some **heap memory**.
- Once the scope of that pointer variable is over, it is removed.
- However, the **memory to which it was pointing remains allocated** till the program execution ends. (till the program execution comes to an end).

Example

```
void disp()
{
	int* ptr = new int(5);
}
int main()
{
	disp();
	// lots of other stuff
}
```

Problem

- The pointer ptr goes out of scope after disp() ends.
- The dynamically allocated memory is **not released**, causing a **memory leak**.

Solution

- Before the end of the disp method, **apply delete on ptr**.

```
void disp()
{
	int* ptr = new int(5);
	delete ptr;
}
```

Dangling Pointer

Definition

- A **dangling pointer** occurs when a pointer is pointing to some **heap memory.**
- After using that memory, we apply **delete** on that pointer to **release** the memory.
- Now, **heap memory is released**, but **the pointer variable still holds the address of the freed memory** till its scope ends.

Example

```
int main()
{
	int* ptr = new int(5);
	// use that ptr pointer
	delete ptr;
	// other stuff
}
```

Problem

- After delete, ptr still contains the old address (dangling pointer).
- So if we perform any operation using that pointer it will give Runtime Error.

Solution

- After applying delete on the pointer, **assign NULL (or nullptr) to it**.

```
delete ptr;
ptr = NULL;  // or ptr = nullptr;
```

Importance of Character Pointer

Example 1: Without const

```
void disp(char* ptr)
{
		cout << *ptr << endl;
}
int main()
{
		disp("hello");  // error
}
```

Explanation:

- The above code will not compile as "hello" is of type **pointer to const** (const char*).
- To collect it, the function parameter must be **pointer to const**.

Corrected Version

```
void disp(const char* ptr)
{
		cout << *ptr << endl;
}
int main()
{
		disp("hello");
}
```

Alternative

If we do not want to declare pointer to constant as the function parameter, the following code will work:

```
void disp(char* ptr)
{
	cout << *ptr << endl;
}
int main()
{
	disp((char*)"hello");
}
```

Using Strings in C++

Example

```
#include <iostream>
using namespace std;
int main()
{
	//char str[20];
	// str = "hello";  // not possible

	char str[20] = "hello";
	cout << str << endl;

	char str1[] = "welcome";
	cout << str1 << endl;

	cout << "enter string without space" << endl;
	char str2[20];
	cin >> str2;
	cout << str2 << endl;

	cout << "enter string with space" << endl;
	char str3[20];
	cin.ignore(numeric_limits<streamsize>::max(), '\n');
	cin.getline(str3, 19);
	cout << str3 << endl;

	char str4[20];
	strcpy_s(str4, strlen("computer") + 1, "computer");
	cout << str4 << endl;

	// char str5[3] = { 'a','b','c'}; // not correct
	char str5[4] = { 'a','b','c' };
	cout << str5 << endl;
}
```

Explanation

- Strings in C++ are arrays of characters terminated by a null character ('\0').
- cin can be used for strings **without spaces**.
- getline() is used for strings **with spaces**.
- strcpy_s() is used to copy one string into another safely.

Copying One String to Another

```
int main()
{
	char src[] = "computer";
	char dest[20];
	strcpy_s(dest, strlen(src) + 1, src);
	cout << dest << endl;
}
```

Mathematical Functions in C++

```
#include <iostream>
#include <cmath>
using namespace std;

int main()
{
	int x = 49;
	cout << "sqrt(" << x << ") = " << sqrt(x) << endl;

	// Rounding functions
	double num = 3.6;
	cout << "ceil(" << num << ") = " << ceil(num) << endl;
	cout << "floor(" << num << ") = " << floor(num) << endl;
	cout << "round(" << num << ") = " << round(num) << endl;
	cout << "trunc(" << num << ") = " << trunc(num) << endl;
	cout << "abs(" << num << ") = " << abs(num) << endl;

	double base = 3.0;
	double exponent = 2.0;
	cout << "pow(" << base << ", " << exponent << ") = " << pow(base, exponent) << endl;
	cout << "fmod(" << base << ", " << exponent << ") = " << fmod(base, exponent) << endl;

	return 0;
}
```

Common Math Functions

Function	Description	Example
sqrt(x)	Square root of x	sqrt(49) → 7
ceil(x)	Rounds up to nearest integer	ceil(3.6) → 4
floor(x)	Rounds down to nearest integer	floor(3.6) → 3
round(x)	Rounds to nearest integer	round(3.6) → 4
trunc(x)	Truncates decimal part	trunc(3.6) → 3
abs(x)	Absolute value	abs(-3.6) → 3.6
pow(a, b)	Power function	pow(3, 2) → 9
fmod(a, b)	Floating-point remainder	fmod(3, 2) → 1

Structure (struct) in C++

1. Introduction

- A **structure** in C++ is a user-defined data type that groups different data items (possibly of different types) under a single name.
- Structures are useful for representing entities such as *Student, Employee,* etc.
- In C++, structures can also contain **member functions**, making them similar to classes.
- The only difference between structure and class is
    - **The instance** members of **Structure** are by default **Public.**
    - **The instance** members of the **Class** are by default **Private.**

---

2. Basic Structure Example

```
#include<iostream>
using namespace std;

struct Student
{
	char name[20] = "noname";
	int age = 0;
};

int main()
{
	Student s1;
	// strcpy_s(s1.name, "abc");
	// s1.age = 16;

	cout << s1.name << "\t" << s1.age << endl;
	s1.age = -5; // risk of data corruption
	cout << s1.name << "\t" << s1.age << endl;
}
```

Explanation:

- Here, Student is a structure containing two members: name and age.
- By default, structure members are **public**.
- Direct modification (e.g., s1.age = -5;) may cause **data corruption**.

---

3. Private Members in Structure

```
#include<iostream>
using namespace std;

struct Student
{
	char name[20] = "noname";
	int age = 0;
};

int main()
{
	Student s1;
	strcpy_s(s1.name, "abc");
	s1.age = 16;
	cout << s1.name << "\t" << s1.age << endl;
	s1.age = -5; // risk of data corruption
```

```
	cout << s1.name << "\t" << s1.age << endl;

}
```

Explanation:

- Structure members can be declared **private**. But are by default public.
- Private members cannot be accessed directly from outside the structure.
- Helps protect data from accidental modification.

---

4. Member Functions in Structure

Member functions act as an interface between data and the program.

```
#include<iostream>
using namespace std;

struct Student
{
private:
	char name[20] = "noname";
	int age = 0;

public:
	void setName(const char* ptr) // member function
	{
		strcpy_s(name, ptr);
	}
	char* getName()
	{
		return name;
	}
	void setAge(int k)
	{
		age = k;
	}
	int getAge()
	{
		return age;
	}
};

int main()
{
	Student s1;
	 indirect access to data
	 s1.setName("abc");
	 s1.setAge(16);

	cout << s1.getName() << "\t" << s1.getAge() << endl;
	 //s1.age = -5;  // Not allowed — No risk of data corruption
}
```

Explanation:

- Member functions are used to access and modify private data members safely.
- Provides **data encapsulation**.
- Similar to getter and setter methods used in classes.

---

5. Structure with Dynamic Memory Allocation

```
#include <iostream>
using namespace std;

struct Student
{
	char name[30] = "noname";
	int age = 0;
};

int main()
{
	struct Student* ptr;
	cout << "How many records?" << endl;
	int rec;
	cin >> rec;

	ptr = new Student[rec];  // Dynamic array of structure

	for (int i = 0; i < rec; i++)
	{
		cin >> ptr[i].name >> ptr[i].age;
	}

	for (int i = 0; i < rec; i++)
	{
		cout << ptr[i].name << "\t" << ptr[i].age << endl;
	}

	// delete ptr; // Warning: destructors won't be called
	delete[] ptr;  // Correct way to free memory
}
```

Explanation:

- Dynamic memory for multiple structure objects can be allocated using new.
- Always release memory using delete[] for arrays to avoid memory leaks.

---

6. Structure with Pointer and Array Access

```
#include <iostream>
using namespace std;

struct Student
{
	char name[30] = "noname";
	int age = 0;
};

int main()
{
	Student s1 = { "Abc", 25 };
	Student* ptr1 = &s1;

	cout << s1.name << "\t" << s1.age << endl;
	cout << ptr1->name << "\t" << ptr1->age << endl;
	cout << (*ptr1).name << "\t" << (*ptr1).age << endl;

	Student arr[3] = { {"xyz", 35}, {"aac", 23}, {"jkl", 31} };
	for (int i = 0; i < 3; i++)
	{
		cout << arr[i].name << "\t" << arr[i].age << endl;
	}

	struct Student* ptr;
```

```
        cout << "How many records?" << endl;
        int rec;
        cin >> rec;
        ptr = new Student[rec];
        for (int i = 0; i < rec; i++)
        {
                cin >> ptr[i].name >> ptr[i].age;
        }
        for (int i = 0; i < rec; i++)
        {
                cout << ptr[i].name << "\t" << ptr[i].age << endl;
        }

        delete[] ptr;
}
```

Explanation:

- Structure variables can be accessed using:
  - . (dot operator) for direct objects.
  - -> (arrow operator) for pointer objects.
- Structures can be stored in arrays and dynamically allocated at runtime.

---

7. Summary Table

Concept	Description	Example / Remark
Public Members	Accessible directly	s1.age = 20;
Private Members	Not accessible directly	Must use setters/getters
Member Functions	Define operations on data	setAge(), getAge()
Pointer to Structure	Access members using ->	ptr->age OR (*ptr).age
Array of Structures	Multiple records	Student arr[3];
Dynamic Allocation	Using new and delete[]	ptr = new Student[n];
Encapsulation	Protects data from corruption	Private data + public interface

---

External vs Global Variables

Global Variable

- A variable defined **outside all functions** is a **global variable**.
- It is accessible **throughout the file** (and by default across files unless restricted).
- Declared at the top of the main function.

External Variable

- The keyword **extern** tells the compiler that:
  - **The variable is defined elsewhere** (in another file, or below in the same file).
  - **No memory** should be allocated for it here — only a **reference** is created.

Example 1 — Declaration without Definition

```
#include<iostream>
using namespace std;
int a = 10;  // Global variable
int main()
{
   extern int b; // External variable declaration
   cout << a << endl;
   cout << b << endl; // Compiles, but linker error
   return 0;
}
// int b = 20;  // Uncommenting this resolves the linker error
```

- The above will compile but give a **linker error** because b is declared but not defined.

Example 2 — Declaration + Definition

```
#include<iostream>
using namespace std;
int a = 10;
int main()
{
   extern int b;
   cout << a << endl;
   cout << b << endl;
   return 0;
}
int b = 20;  // Definition of b
```

- Now the linker resolves b successfully.

Example 3 — Access without Declaration

```
#include<iostream>
using namespace std;
int a = 10;
int main()
{
   cout << a << endl;
   cout << b << endl; // Error: 'b' undeclared identifier
   return 0;
}
int b = 20;
```

Linkage

Only for Global or Extern Variables Functions;
Not For Local Variables.

- Linkage in C++ defines **how variable or function names refer to storage**.
- Describe how variables & Functions should be linked by linker.
- Whether they are visible across files or only within a single translation unit (source file).

Types of Linkage

Type	Meaning	Keyword / Default	Scope
External Linkage	Identifier is visible across files	extern or default for globals	Accessible from other files
Internal Linkage	Identifier is visible only within one file	static	Restricted to current file
Type-safe Linkage	Ensures linkage matches type signatures	(automatic in C++)	Detects mismatched declarations
Alternate Linkage	Allows C functions to be used in C++	extern "C"	Removes name mangling

---

1. External Linkage

- **External linkage** means **a single storage** is created for a symbol, shared across multiple files.
- Variables or functions with external linkage can be accessed from other files using **extern**.
- By default, global variables and functions have **external linkage**.
- Compiler writes the instruction to the linker.

Simple Definition

Global variables can be accessed outside the file — this is **External Linkage**.

Practical Example (Two-File Project)

Step 1: Project 1 → External_1

File: First.cpp

```
#include<iostream>
using namespace std;
void disp()
{
   extern int num;
```

```
  cout << "in disp\t" << num << endl;
  void fun(); // Implicitly extern
  cout << "from disp calling fun\t";
  fun();
}
```

- **Compile only** (Ctrl + F7).

Step 2: Project 2 → External_2

File: Second.cpp

```
#include<iostream>
using namespace std;
#include "C:\\MyCpp\\External_1\\First.cpp"

int num = 100;
void fun()
{
  cout << "in fun" << endl;
}
int main()
{
  cout << "in main\t" << num << endl;
  disp();
  cout << "from main calling fun\t";
  fun();
}
```

- **Compile and build.**
- The executable will run successfully as External_2.exe.

---

2. Internal Linkage

- **Internal linkage** means a variable or function is **visible only within the current file(Translation unit)** in which they are defined.
- Declared using the static keyword at global scope.
- Other files may use the same name, but each will have its **own storage**.
- Other cant access them.

Simple Definition

Static global variables or functions cannot be accessed outside the file — this is **Internal Linkage**.

Practical Example

Project 1 → Internal_1

File: First.cpp

```
#include<iostream>
using namespace std;
void disp()
{
```

```
  extern int num;
  cout << "in disp\t" << num << endl;
  void fun();
  cout << "from disp calling fun\t";
  fun();
}
```

Compile only (Ctrl + F7).

Project 2 → Internal_2

File: Second.cpp

```
#include<iostream>
using namespace std;
#include "C:\\MyCpp\\Internal_1\\First.cpp"

static int num = 100;
static void fun()
{
  cout << "in fun" << endl;
}
int main()
{
  cout << "in main\t" << num << endl;
  disp();
  cout << "from main calling fun\t";
  fun();
}
```

- Compilation succeeds, but **linker error** occurs: "**unresolved external symbols: num, fun**". Because static gives **internal linkage**, making them invisible to other files.

Behavior of static and extern

Keyword	Linkage Type	Visible to Other Files	Symbol Type
int cnt = 100;	External	Yes	Global
static int cnt = 100;	Internal	No	Local

---

3. Type-safe Linkage

- When you define library function taking an int, but somewhere you call it using string.
- Hence In **C**, the linker only matches **function names**, not signatures. So, mismatched declarations **compile and linking** will be done successfully.
- but cause **runtime errors**.

C Example

Library

```
#include<stdio.h>
void disp(char* ptr)
{
  printf("%c\n", *ptr);
}
```

Client

```
#include<stdio.h>
void disp(double);
void main()
{
  puts("before");
  disp(23.7);
  puts("after");
}
```

- Compiles and links (because the linker matches only the function name disp).
- **Runtime error** occurs because the types mismatch.

In C++ (Type-safe Linkage)

- In C++, **function name mangling** prevents such mismatches.
- Here Compiler mangles the function name with its arguments.
- So here linker look for the mangled name and give linking error instead of Runtime error.

C++ Library

```
#include<iostream>
using namespace std;
void disp(char* ptr)
{
  cout << *ptr << endl;
}
```

Client

```
#include<iostream>
void disp(double);
using namespace std;
void main()
{
  cout << "before" << endl;
  disp(4.5);
  cout << "after" << endl;
}
```

- C++ compiler **decorates** names based on parameter types.
  - void disp(char*) → internally disp_char*
  - void disp(double) → internally disp_double
- The linker cannot match disp_double with disp_char* → **link error**, not runtime error.
- This is called **type-safe linkage**.

---

4. Alternate Linkage (C with C++)

- Also known as language linkage.
- When C++ uses a **C library**, name mangling causes linking failure because the **C compiler does not mangle names**.

Example

C Library Function

```
float fun1(int, char);
```

C++ Declaration

```
float fun1(int, char);
```

- The C++ compiler mangles it as **fun1_int_char.**
- The C library defines it as plain **fun1.**
- **Linker error** occurs.

Solution — extern "C"

```
extern "C" float fun1(int, char);
```

- Instructs the compiler **not to mangle the name**.
- Enables correct linking with C libraries.

Practical Example

Step 1: Developer Library (C)

Project: devpro (Static Library)
Location: C:\MyCpp\developer

Header: Header.h

```
void disp(int);
```

Source: devpro.c

```
#include<stdio.h>
void disp(int k)
{
   printf("Inside disp\t%d\n", k);
}
```

- Build to generate:
  C:\MyCpp\developer\devpro\x64\Debug\devpro.lib
- Copy both devpro.lib and Header.h to C:\forclient.

Step 2: Client Program (C++)

Project: clientpro
File: ClientApp.cpp

```
#include<iostream>
#include "C:\\forclient\\Header.h"
using namespace std;
int main()
{
   cout << "Before calling" << endl;
   disp(100);
   cout << "After calling" << endl;
}
```

Problem:

- Compilation succeeds.
- Linking fails — C++ mangles function name (disp_int) while devpro.lib contains plain disp.

Step 3: Solution — Use extern "C" in Header

Updated Header.h

```
#ifdef __cplusplus              //__cplusplus is the macro defined only in cpp
extern "C" {
#endif

            void disp(int);

#ifdef __cplusplus
}
#endif
```

- Rebuild both developer and client projects.
- Linking will succeed and program executes correctly.
